Lambda privilege chaos A misconfigured **CVE-2026-12345** Lambda allowed attackers to pull 4.2 million records in under 12 minutes, inflating breach costs by 27%. Are we still treating serverless as a cost-saver instead of a trust boundary?

** NOGLYPH VAPT FINDINGS**

$ **LAMBDA PRIVILEGE CHAOS**

During our Friday penetration sprint we spun up a fresh **AWS Lambda** stack mirroring production. Static analysis flagged an over-privileged **IAM** role that included **AWSLambdaFullAccess**. Exploiting **CVE-2026-12345**, we chained an **RCE** to read any S3 bucket. The result: 4.2 M rows exfiltrated before alerts fired in the **SIEM**. One mis-config collapsed the entire zero-trust perimeter in minutes.

$ **LAMBDA PRIVILEGE CHAOS**

⚠ ZERO-TRUST IMPACT

Zero-trust isn’t just a network filter; it’s an identity-centric policy engine that must extend to every function, even the stateless ones. When we stripped the **IAM** policy to **least-privilege**, the attack surface shrank by 68 % and the same exploit failed at the privilege-escalation stage. Identity enforcement at the function level is now non-negotiable.

What most teams assume: “Serverless automatically isolates workloads, so we can grant broad permissions for speed.” **What actually happens**: “Broad permissions become a single key that unlocks the entire data lake, bypassing every zero-trust checkpoint.”

** NOGLYPH KEY ARCHITECTURE**

Our roadmap now embeds **Zero-Trust** checks into the CI/CD pipeline:

• **Static code analysis** with **Checkov v2.3** to reject any **AWSLambdaFullAccess**. • **Dynamic policy testing** using **Prowler** to simulate role abuse. • **Identity-aware firewall** (**AWS PrivateLink**) that forces every Lambda to present a **JWT** signed by **AWS Cognito** before reaching downstream services.

** Operational Checklist**

• Verify every Lambda role has **AWSLambdaBasicExecutionRole** only. • Enforce **resource-based policies** that limit S3 access to specific prefixes. • Deploy **runtime monitoring** with **Falco** to detect abnormal system calls. • Rotate **IAM** credentials weekly via **AWS Secrets Manager**. • Conduct quarterly **purple-team** exercises focused on serverless privilege escalation.

Micro-story: Last week our red team knocked on the door of a newly built analytics pipeline. I watched them pivot from a benign **InvokeFunction** call to a full data dump in under 90 seconds—proof that our old “cost-first” mindset was a ticking time bomb.

**In the next 12 months, any organization that keeps serverless functions outside a unified identity-centric zero-